



Engineering
Accreditation
Commission



Departamento de Ingeniería de Sistemas y Computación
Estructuras de Datos y Algoritmos
ISIS-1225

ANÁLISIS DEL RETO 04

Gabriel Andrés Moya Caravaca - 202616964 - g.moyac@uniandes.edu.co

Gerardo Rocha Linares - 202513388 - g.rochal2@uniandes.edu.co

Edgar Daniel Gonzalez Martinez - 202524741 - ed.gonzalezml@uniandes.edu.co

Requerimiento 1

Descripción

Entrada	Identificador de la zona de navegación de origen (DEST_CLUSTER). Identificador de la zona de navegación de destino (DEST_CLUSTER).
Salidas	La ruta con menos conexiones entre dos zonas de navegación, mostrando las zonas recorridas y su información.
Implementado	Si, fue implementado por Gabriel Andrés Moya Caravaca.

Análisis

El requerimiento 1 encuentra la trayectoria con el menor número de arcos entre dos zonas de navegación. El usuario ingresa el identificador de la zona origen y el identificador de la zona destino, ambos como DEST_CLUSTER. Primero se valida que ambas zonas existan en el grafo. Luego se ejecuta BFS desde la zona origen, que recorre el grafo nivel por nivel, visitando cada vértice una sola vez y explorando todos sus arcos. Esto tiene un costo de $O(V + E)$, donde V es el número de zonas y E el número de arcos. Una vez terminado el BFS, se reconstruye el camino con `path_to`, que sigue los `edge_from` desde el destino hasta el origen y los agrega al frente de un `array_list`, resultando en $O(V)$ en el peor caso. Finalmente se seleccionan los vértices a mostrar, que son todos si el camino tiene 10 o menos zonas, o los primeros 5 y últimos 5 si tiene más, lo cual es $O(1)$. La complejidad total del requerimiento es $O(V + E)$, dominada por el BFS.

```
# Funciones de consulta sobre el catálogo
def req_1(catalog, zona_origen, zona_destino):
    """
    Retorna el resultado del requerimiento 1
    """
    # TODO: Modificar el requerimiento 1
    resultado = {
        "existe": False,
        "total_zonas": 0,
        "vertices": al.new_list(),
        "origen_ok": digraph.contains_vertex(catalog["graph"], zona_origen),
        "destino_ok": digraph.contains_vertex(catalog["graph"], zona_destino),
    }

    if not resultado["origen_ok"] or not resultado["destino_ok"]:
        return resultado

    if zona_origen == zona_destino:
        resultado["existe"] = True
        resultado["total_zonas"] = 1
        info = digraph.get_vertex_info(catalog["graph"], zona_origen)
        al.add_last(resultado["vertices"], vertex_summary(info))
        return resultado

    # BFS desde el origen
    visited_map = bfs_module.bfs(catalog["graph"], zona_origen)

    if not bfs_module.has_path_to(zona_destino, visited_map):
        return resultado
```

```
# Reconstruir camino con path_to
path_stack = bfs_module.path_to(zona_destino, visited_map)
path = al.new_list()
from DataStructures.Stack import stack as st
while not st.is_empty(path_stack):
    al.add_last(path, st.pop(path_stack))

total = al.size(path)
resultado["existe"] = True
resultado["total_zonas"] = total

# Índices a mostrar primeros 5 y últimos 5
if total <= 10:
    for i in range(total):
        zona_key = al.get_element(path, i)
        info_v = digraph.get_vertex_info(catalog["graph"], zona_key)
        al.add_last(resultado["vertices"], vertex_summary(info_v))
else:
    for i in range(5):
        zona_key = al.get_element(path, i)
        info_v = digraph.get_vertex_info(catalog["graph"], zona_key)
        al.add_last(resultado["vertices"], vertex_summary(info_v))
    for i in range(total - 5, total):
        zona_key = al.get_element(path, i)
        info_v = digraph.get_vertex_info(catalog["graph"], zona_key)
        al.add_last(resultado["vertices"], vertex_summary(info_v))

return resultado
```

Requerimiento 2

Descripción

El Requerimiento 2 se encarga de encontrar las zonas de navegación que están dentro de un radio específico alrededor de una zona de interés. Para esto, el usuario ingresa el identificador de la zona, es decir el DEST_CLUSTER, y también el radio en kilómetros que quiere revisar. Primero se valida que la zona exista en el grafo. Luego se obtiene la latitud y longitud representativa de esa zona. Después, el algoritmo recorre todos los vértices del grafo, porque cada vértice representa una zona de navegación. Para cada una de esas zonas se calcula la distancia respecto a la zona de origen usando la fórmula de Haversine. Si esa distancia es menor o igual al radio ingresado, entonces la zona se agrega a la lista de resultados. Al final, las zonas encontradas se ordenan por distancia ascendente, o sea, desde la más cercana hasta la más lejana. Si dos zonas tienen la misma distancia, se ordenan por el identificador de la zona.

Entrada	*Identificador de la zona de navegación de interés, por ejemplo: "1242". *Radio del área de interés en kilómetros, por ejemplo: 70.
Salidas	El requerimiento muestra el total de zonas que quedaron dentro del área de interés. Para cada zona se muestra su identificador, latitud, longitud, número total de registros asociados y velocidad promedio. También se puede mostrar la distancia en kilómetros, aunque el enunciado no la pide directamente, porque ayuda a comprobar que el ordenamiento sí quedó bien.
Implementado	Sí, fue implementado por Gerardo Rocha Linares.

Análisis de complejidad

La complejidad del Requerimiento 2 depende del recorrido completo de los vértices del grafo y del ordenamiento final de las zonas encontradas. Primero se valida si la zona de interés existe con `digraph.contains_vertex`, lo cual cuesta $O(1)$ promedio porque el grafo trabaja internamente con mapas. Luego se obtiene la información del vértice origen con `digraph.get_vertex_info`, también en $O(1)$ promedio. Después se obtiene la lista de vértices con `digraph.vertices`, lo cual cuesta $O(V)$, donde V es el número total de zonas del grafo. Luego se recorren todos esos vértices, ya que el enunciado exige evaluar cada zona del grafo completo. En cada iteración se consulta la información del vértice, se calcula la distancia con Haversine, se compara con el radio y, si cumple, se agrega a la lista de resultados; todas estas operaciones son $O(1)$, por lo que el recorrido cuesta $O(V)$. Finalmente, si r es la cantidad de zonas dentro del radio, estas se ordenan con merge sort en $O(r \log r)$, usando como criterio la distancia ascendente y luego el identificador de la zona. Por eso, la complejidad final es $O(V + r \log r)$. En el peor caso, si todas las zonas quedan dentro del radio, $r = V$ y la complejidad sería $O(V \log V)$.

Análisis

```
def req_2(catalog, zona_origen, radio):  
    """  
    Retorna el resultado del requerimiento 2.  
    Detecta las zonas de navegación dentro de un radio alrededor  
    de una zona de navegación de interés.  
    """  
  
    resultado = {  
        "zona_ok": digraph.contains_vertex(catalog["graph"], zona_origen),  
        "total_zonas": 0,  
        "zonas": al.new_list()  
    }  
  
    if not resultado["zona_ok"]:  
        return resultado  
  
    info_origen = digraph.get_vertex_info(catalog["graph"], zona_origen)  
  
    lat_origen = info_origen["lat"]  
    lon_origen = info_origen["lon"]  
  
    vertices = digraph.vertices(catalog["graph"])  
  
    for i in range(al.size(vertices)):  
        zona_actual = al.get_element(vertices, i)  
        info_zona = digraph.get_vertex_info(catalog["graph"], zona_actual)  
  
        distancia = distancia_haversine(  
            lat_origen,  
            lon_origen,  
            info_zona["lat"],  
            info_zona["lon"]  
        )  
  
        if distancia <= radio:  
            resumen = vertex_summary_req_2(info_zona, distancia)  
            al.add_last(resultado["zonas"], resumen)  
  
    al.merge_sort(resultado["zonas"], cmp_req_2)  
  
    resultado["total_zonas"] = al.size(resultado["zonas"])  
  
    return resultado
```

La implementación del código es la ideal para este requerimiento, ya que el objetivo no es encontrar una ruta entre zonas, sino identificar cuáles zonas están dentro de un radio geográfico respecto a una zona de interés. Por eso, el algoritmo trabaja directamente sobre los vértices del grafo, donde cada vértice representa una zona de navegación, y revisa uno por uno para calcular su distancia con la fórmula de Haversine. Este recorrido es necesario porque una zona puede estar cerca geográficamente aunque no esté conectada directamente por un arco con la zona origen. Después de calcular las distancias, el código filtra las zonas que cumplen con el radio indicado y las ordena por distancia ascendente, usando el identificador del vértice como criterio de desempate. De esta forma, la solución cumple con las especificaciones sin usar estructuras innecesarias ni recorridos como BFS, DFS o Dijkstra, que en este caso no aportarían porque el problema se basa en cercanía geográfica y no en caminos dentro del grafo.

Requerimiento 3

Descripción

Entrada	Parámetros necesarios para resolver el requerimiento.
Salidas	Respuesta esperada del algoritmo.
Implementado	Si, fue implementado por Edgar Daniel Gonzalez Martinez.

Análisis de complejidad

El requerimiento 3 tiene una complejidad de $O(E \log E)$. Arranca con un BFS que recorre todo el grafo visitando cada zona y cada arco una sola vez, eso cuesta $O(V + E)$. Después viene el Merge Sort que toma todos los arcos encontrados y los ordena dividiéndolos a la mitad una y otra vez hasta tener pedazos de un solo elemento, y eso cuesta $O(E \log E)$. Al final solo se sacan los primeros N arcos para mostrarlos, que es $O(N)$ y no pesa nada comparado con lo anterior. El que manda en el costo total es el Merge Sort porque $O(E \log E)$ crece más rápido que $O(V + E)$, así que la complejidad final del requerimiento es $O(E \log E)$.

Análisis

El REQ 3 empieza sacando todas las zonas del grafo y haciendo un BFS desde cada zona que no haya sido visitada. Por cada zona que sale de la cola revisa todos sus vecinos, construye la llave juntando el nombre de la zona actual con el del vecino, busca el arco en el mapa de arcos y si lo encuentra lo guarda en la lista. Si el vecino no había sido visitado lo mete a la cola. Así se garantiza pasar por todo el grafo y recoger todos los arcos una sola vez sin repetir ninguno.

Con todos los arcos ya reunidos se llama al merge sort que los va partiendo a la mitad una y otra vez hasta tener pedazos de un solo elemento, y luego los va uniendo comparando cuál tiene más viajes, si empatan mira cuál tiene el origen más pequeño, y si siguen empatados desempata por el destino.

Con la lista ordenada simplemente toma los primeros N arcos y por cada uno verifica que los campos tengan valor, si alguno no tiene le pone "Unknown", y los valores de distancia y tiempo los redondea a dos decimales antes de meterlos en el resultado final.

```
def req_3(catalog,n):
    """
    Retorna el resultado del requerimiento 3
    """
    # TODO: Modificar el requerimiento 3

    grafo = catalog["graph"]
    visitados = set()
    lista_arcos = []
    cola = []
    todas_las_zonas = digraph.vertices(grafo)
    for i in range(al.size(todas_las_zonas)):
        zona_inicio = al.get_element(todas_las_zonas, i)

        if zona_inicio not in visitados:
            visitados.add(zona_inicio)
            cola.append(zona_inicio)

            while cola:
                zona_actual = cola.pop(0)
                vecinos = digraph.adjacents(grafo, zona_actual)
                for j in range(len(vecinos)):
                    vecino = vecinos[j]
                    llave_arco = zona_actual + "_" + vecino
                    arco = mp.get(catalog["edge_info_map"], llave_arco)

                    if arco is not None:
                        lista_arcos.append(arco)

                    if vecino not in visitados:
                        visitados.add(vecino)
                        cola.append(vecino)

    lista_ordenada = merge_sort_arcos(lista_arcos)

    resultado = al.new_list()

    for arco in lista_ordenada[:n]:
        origen = arco["source"] if arco["source"] else "Unknown"
        destino = arco["target"] if arco["target"] else "Unknown"
        viajes = arco["trips_count"] if arco["trips_count"] else "Unknown"

        if arco["distance"] is not None:
            distancia = round(arco["distance"], 2)
        else:
            distancia = "Unknown"

        if arco["avg_time"] is not None:
            tiempo_promedio = round(arco["avg_time"], 2)
        else:
            tiempo_promedio = "Unknown"

        al.add_last(resultado, {
            "source": origen,
            "target": destino,
            "trips_count": viajes,
            "distance": distancia,
            "avg_time": tiempo_promedio,
        })

    return resultado
```

Requerimiento 4

Descripción

Entrada	Zona de navegación de origen.
Salidas	La red resultante corresponde al conjunto de arcos que definen los caminos mínimos desde la zona de origen hacia las demás zonas alcanzables.
Implementado	Si, fue implementado por G07

Análisis

El requerimiento 4 construye el árbol de caminos mínimos desde una zona de navegación origen hacia todas las demás zonas alcanzables. El usuario ingresa únicamente el identificador de la zona origen como DEST_CLUSTER. Primero se valida que la zona exista en el grafo. Luego se ejecuta Dijkstra con un min-heap, que procesa cada vértice una vez extrayéndolo de la cola de prioridad con costo $O(\log V)$, y por cada arco puede actualizar la distancia de un vecino con `improve_priority`, también $O(\log V)$. Esto da una complejidad de $O((V + E) \log V)$. Una vez terminado Dijkstra, se recorre el `visited_map` para extraer los arcos del SPT, es decir el árbol de caminos mínimos, revisando el `edge_from` de cada vértice marcado, con costo $O(V)$. Luego se ordenan los arcos con merge sort, que sobre los $V-1$ arcos del SPT tiene costo $O(V \log V)$. Finalmente se calcula el costo total recorriendo todos los arcos en $O(V)$ y se seleccionan los primeros 5 y últimos 5 para mostrar en $O(1)$. La complejidad total del requerimiento es **$O((V + E) \log V)$** , dominada por Dijkstra.

```
def req_4(catalog, zona_origen):
    """
    Retorna el resultado del requerimiento 4
    """
    # TODO: Modificar el requerimiento 4
    resultado = {
        "origen_ok": digraph.contains_vertex(catalog["graph"], zona_origen),
        "total_zonas": 0,
        "costo_total": 0.0,
        "arcos": al.new_list(),
        "arcos_tabla": al.new_list(),
    }

    if not resultado["origen_ok"]:
        return resultado

    # Dijkstra
    aux = bfs_module.dijkstra(catalog["graph"], zona_origen)
    visited_map = aux["visited"]

    # Recorrer visited_map para extraer arcos del SPT
    llaves = mp.key_set(visited_map)
    for i in range(al.size(llaves)):
        key_v = al.get_element(llaves, i)
        info = mp.get(visited_map, key_v)
        if info["marked"] and info["edge_from"] is not None:
            origen_arco = info["edge_from"]
            dist_destino = info["dist_to"]
            dist_origen = mp.get(visited_map, origen_arco)["dist_to"]
            peso = round(dist_destino - dist_origen, 2)

            arco = {
                "origen": origen_arco if origen_arco is not None else "Unknown",
                "destino": key_v if key_v is not None else "Unknown",
                "peso": peso,
            }
            al.add_last(resultado["arcos"], arco)
            if info["marked"]:
                resultado["total_zonas"] += 1

    # Ordenar arcos por origen asc, en empate por destino asc
    sort_arcos(resultado["arcos"])

    # Costo total
    costo = 0.0
    for i in range(al.size(resultado["arcos"])):
        costo += al.get_element(resultado["arcos"], i)["peso"]
    resultado["costo_total"] = round(costo, 2)

    # Primeros 5 y últimos 5 arcos
    total_arcos = al.size(resultado["arcos"])
    if total_arcos <= 10:
        for i in range(total_arcos):
            al.add_last(resultado["arcos_tabla"], al.get_element(resultado["arcos"], i))
    else:
        for i in range(5):
            al.add_last(resultado["arcos_tabla"], al.get_element(resultado["arcos"], i))
        for i in range(total_arcos - 5, total_arcos):
            al.add_last(resultado["arcos_tabla"], al.get_element(resultado["arcos"], i))

    return resultado
```

Requerimiento 5

Descripción

El Requerimiento 5 se encarga de encontrar la ruta más eficiente entre dos zonas de navegación. Para esto, el usuario ingresa el identificador de una zona de origen y el identificador de una zona de destino. Como el grafo es dirigido y sus arcos tienen un peso asociado a la distancia entre zonas, se utiliza el algoritmo de Dijkstra para encontrar el camino de menor costo desde el origen hasta el destino. Primero se valida que tanto la zona de origen como la zona de destino existan dentro del grafo. Luego se ejecuta Dijkstra desde la zona de origen, usando los pesos de los arcos del grafo. Si la zona destino fue alcanzada, se reconstruye la ruta usando la información de los vértices anteriores guardados por el algoritmo. Después se calcula el costo total de la ruta, el número total de zonas conectadas y el número total de arcos recorridos. Finalmente, se muestran los vértices de la ruta en orden desde el origen hasta el destino. Si la ruta tiene más de 10 vértices, solo se muestran los cinco primeros y los cinco últimos.

Entrada	*Identificador de la zona de navegación de origen, por ejemplo: "1242". *Identificador de la zona de navegación de destino, por ejemplo: "1129".
Salidas	El requerimiento muestra si existe o no una ruta entre las zonas ingresadas. En caso de existir, se muestra el costo total de la ruta, el número total de zonas conectadas, el número total de arcos en la ruta y una tabla con los vértices correspondientes. Para cada vértice se muestra el identificador de la zona, la latitud y longitud representativas, el número de embarcaciones que han transitado por esa zona y el peso del arco hacia el siguiente vértice. Si no existe una ruta entre las dos zonas, se muestra un mensaje indicándolo claramente. En caso de que algún valor no exista, se reporta como "Unknown".
Implementado	Si, fue implementado por G07

Análisis de complejidad

La complejidad del Requerimiento 5 depende principalmente de la ejecución de Dijkstra, porque se busca la ruta de menor costo entre una zona origen y una zona destino dentro de un grafo dirigido con pesos. Primero se valida si ambos vértices existen con `digraph.contains_vertex`, lo cual cuesta $O(1)$ promedio porque el grafo maneja sus vértices mediante mapas. Luego se ejecuta Dijkstra desde la zona origen; si V es el número de vértices del grafo y E el número de arcos, esta parte cuesta aproximadamente $O((V + E) \log V)$, considerando el uso de una cola de prioridad. Después se consulta en el mapa de visitados si el destino fue alcanzado, lo cual cuesta $O(1)$ promedio. Si existe ruta, se reconstruye el camino usando `edge_from`; si k es la cantidad de vértices que componen la ruta final, esta reconstrucción cuesta $O(k)$. Luego se recorre esa ruta para obtener la información de cada vértice y calcular el peso del arco hacia el siguiente vértice, lo cual también cuesta $O(k)$. Por eso, la complejidad final es $O((V + E) \log V + k)$. Como k normalmente es menor o igual que V , el costo dominante es Dijkstra, así que la complejidad general puede expresarse como $O((V + E) \log V)$.

Análisis

```
def req_5(catalog, zona_origen, zona_destino):

    resultado = {
        "origen_ok": digraph.contains_vertex(catalog["graph"], zona_origen),
        "destino_ok": digraph.contains_vertex(catalog["graph"], zona_destino),
        "existe": False,
        "costo_total": 0.0,
        "total_zonas": 0,
        "total_arcos": 0,
        "vertices": al.new_list()
    }

    if not resultado["origen_ok"] or not resultado["destino_ok"]:
        return resultado

    if zona_origen == zona_destino:
        resultado["existe"] = True
        resultado["costo_total"] = 0.0
        resultado["total_zonas"] = 1
        resultado["total_arcos"] = 0

        info = digraph.get_vertex_info(catalog["graph"], zona_origen)
        resumen = vertex_summary_req_5(info, "Unknown")
        al.add_last(resultado["vertices"], resumen)

        return resultado

    aux = bfs_module.dijkstra(catalog["graph"], zona_origen)
    visited_map = aux["visited"]

    info_destino = mp.get(visited_map, zona_destino)

    if info_destino is None or not info_destino["marked"]:
        return resultado

    camino_inverso = al.new_list()
    actual = zona_destino
    ruta_valida = True

    while actual != zona_origen and actual is not None:
        al.add_last(camino_inverso, actual)

        info_actual = mp.get(visited_map, actual)

        if info_actual is None:
            ruta_valida = False
            actual = None
        else:
            actual = info_actual["edge_from"]

    if actual is None or not ruta_valida:
        return resultado

    al.add_last(camino_inverso, zona_origen)

    camino = al.new_list()

    for i in range(al.size(camino_inverso) - 1, -1, -1):
        al.add_last(camino, al.get_element(camino_inverso, i))

    total_zonas = al.size(camino)

    resultado["existe"] = True
    resultado["costo_total"] = round(info_destino["dist_to"], 2)
    resultado["total_zonas"] = total_zonas
    resultado["total_arcos"] = total_zonas - 1

    vertices_completos = al.new_list()

    for i in range(total_zonas):
        zona_actual = al.get_element(camino, i)
        info_zona = digraph.get_vertex_info(catalog["graph"], zona_actual)

        if i < total_zonas - 1:
            zona_siguiente = al.get_element(camino, i + 1)
            dist_actual = mp.get(visited_map, zona_actual)["dist_to"]
            dist_siguiente = mp.get(visited_map, zona_siguiente)["dist_to"]

            peso_siguiente = round(dist_siguiente - dist_actual, 2)
        else:
            peso_siguiente = "Unknown"

        resumen = vertex_summary_req_5(info_zona, peso_siguiente)
        al.add_last(vertices_completos, resumen)

    if total_zonas <= 10:
        for i in range(total_zonas):
            al.add_last(resultado["vertices"], al.get_element(vertices_completos, i))
    else:
        for i in range(5):
            al.add_last(resultado["vertices"], al.get_element(vertices_completos, i))

        for i in range(total_zonas - 5, total_zonas):
            al.add_last(resultado["vertices"], al.get_element(vertices_completos, i))

    return resultado
```

La implementación del código es óptima para este requerimiento, ya que el objetivo es encontrar la ruta de menor costo entre una zona de origen y una zona de destino dentro de un grafo dirigido con pesos. Por eso se usa Dijkstra, porque este algoritmo permite calcular caminos mínimos cuando los arcos tienen costos asociados, en este caso la distancia entre zonas de navegación. No tendría sentido usar BFS, porque BFS solo minimiza la cantidad de arcos y no el costo total de la ruta. Tampoco sería adecuado usar DFS, porque solo explora caminos posibles, pero no garantiza que el camino encontrado sea el de menor distancia. Además, la implementación aprovecha la información que ya está en el grafo. Cada vértice representa una zona de navegación y cada arco representa una transición entre zonas con un peso asociado. Después de ejecutar Dijkstra, el código reconstruye únicamente la ruta necesaria entre el origen y el destino, en vez de mostrar toda la red de caminos mínimos. Esto hace que la salida sea más clara y se ajuste a lo que pide el requerimiento. También se conserva el orden correcto de la ruta, desde la zona origen hasta la zona destino, y se reporta el peso del arco hacia el siguiente vértice, lo cual permite entender cómo se va acumulando el costo de la ruta.

Requerimiento 6

Descripción

Entrada	Parámetros necesarios para resolver el requerimiento.
Salidas	Respuesta esperada del algoritmo.
Implementado	Si, fue implementado por G07

Análisis de complejidad

Recorrer todos los arcos para armar la adyacencia cuesta $O(E)$. Ordenar las zonas con merge sort cuesta $O(V \log V)$. El BFS que encuentra todas las componentes pasa por cada zona y cada arco una sola vez así que cuesta $O(V + E)$. Ordenar las componentes con merge sort en el peor caso cuesta $O(V \log V)$. Y el recorrido final para calcular registros y velocidad recorre todas las zonas una vez más costando $O(V)$. Como hay muchos más arcos que vértices el término que manda es $O(E)$, así que la complejidad total del requerimiento queda en $O(V + E)$.

Análisis

El REQ 6 arranca recorriendo todos los arcos del mapa y por cada uno busca si existe su inverso, si los dos existen agrega la conexión en ambas direcciones al diccionario de adyacencia. Así queda armado el grafo no dirigido solo con las zonas que se conectan en los dos sentidos. Después ordena todas las zonas con merge sort para que el BFS siempre arranque en el mismo orden. Luego hace un BFS desde cada zona que no haya sido visitada, va sacando zonas de la cola, las agrega a la componente actual y mete a la cola a todos sus vecinos que no hayan sido visitados, cuando la cola queda vacía esa componente está lista. Con todas las componentes encontradas las ordena de mayor a menor con otro merge sort desempataando por el identificador más pequeño. Al final recorre cada componente sumando los registros y calculando el promedio de velocidad de todas sus zonas.

```
def req_6(catalog):
    grafo = catalog["graph"]
    todas_las_zonas = digraph.vertices(grafo)
    adyacencia = {}

    for i in range(al.size(todas_las_zonas)):
        zona = al.get_element(todas_las_zonas, i)
        adyacencia[zona] = []
    llaves_arcos = mp.key_set(catalog["edge_info_map"])
    for i in range(al.size(llaves_arcos)):
        llave = al.get_element(llaves_arcos, i)
        arco = mp.get(catalog["edge_info_map"], llave)
        origen = arco["source"]
        destino = arco["target"]
        llave_inversa = destino + "_" + origen
        arco_inverso = mp.get(catalog["edge_info_map"], llave_inversa)

        if arco_inverso is not None:
            if destino not in adyacencia[origen]:
                adyacencia[origen].append(destino)
            if origen not in adyacencia[destino]:
                adyacencia[destino].append(origen)

    visitados = set()
    componentes = []
    todas_ordenadas = merge_sort_zonas(list(adyacencia.keys()))

    for zona_inicio in todas_ordenadas:
        if zona_inicio not in visitados:
            componente = []
            cola = []
            visitados.add(zona_inicio)
            cola.append(zona_inicio)

            while cola:
                zona_actual = cola.pop(0)
                componente.append(zona_actual)

                for vecino in adyacencia[zona_actual]:
                    if vecino not in visitados:
                        visitados.add(vecino)
                        cola.append(vecino)

            componentes.append(merge_sort_zonas(componente))
    componentes = merge_sort_componentes(componentes)
    resultado = al.new_list()

    for idx in range(len(componentes)):
        componente = componentes[idx]
        total_registros = 0
        suma_sog = 0.0
        for zona in componente:
            info = digraph.get_vertex_info(grafo, zona)
            total_registros += info["records_count"]
            suma_sog += info["avg_sog"]
        avg_sog_subred = round(suma_sog / len(componente), 2)

        al.add_last(resultado, {
            "id_subred": idx + 1,
            "total_zonas": len(componente),
            "zonas": componente,
            "total_registros": total_registros,
            "avg_sog": avg_sog_subred,
        })

    return resultado
```